

Dental Management System

Project Documentation

1. What Was Built

A full-stack dental clinic management web application (MERN stack) with a Node.js/Express REST API backend, a MongoDB database, and a React frontend. It has two portals: a role-based **Staff Portal** to manage patients, dentists, appointments, a treatment price list, billing, and medical records with file uploads; and a separate self-service **Patient Portal** where patients register/log in independently to view their own appointment history, payment history, and medical reports, download PDFs, and receive automatic emails (booking confirmations, payment receipts, and appointment reminders).

2. Tech Stack & Why

Layer	Technology	Why it was chosen
Frontend	React (Vite) + Tailwind CSS	Fast dev server, component reuse, quick professional styling
Routing	React Router	Client-side navigation across public and protected pages
Charts	Recharts	Revenue trend line chart and appointment-status pie chart on the dashboard
Calendar	date-fns	Powers the month-view appointment calendar
PDF Export	jsPDF + jspdf-autotable	Generates a downloadable PDF invoice client-side, no server round trip
HTTP Client	Axios	API calls with the auth token auto-attached via an interceptor
Backend	Node.js + Express	Lightweight REST API, modular route files per resource
Database	MongoDB + Mongoose	Flexible schema for clinical records; validation & relations via refs
Auth & Roles	JWT + bcrypt	Stateless auth token; passwords hashed; admin vs staff permissions enforced server-side
File Uploads	Multer	Handles X-ray/document uploads attached to medical records
CSV Export	json2csv	Streams the patient list as a downloadable CSV
Emails	Nodemailer	Sends registration, booking, receipt, and reminder emails via SMTP
Scheduled Jobs	node-cron	Runs an hourly background job to send appointment reminder emails
Server-side PDFs	PDFKit	Generates receipt/medical-report PDFs used for both downloads and email attachments

3. Features Implemented — Staff Portal

- Public landing page + staff login/register (JWT auth, bcrypt password hashing)
- Role-based access control: Admin vs Staff, enforced in the API (not just hidden in the UI)
- Dashboard: stat cards, a 6-month revenue trend line chart, an appointment-status pie chart, and a live 'today's appointments' list
- Patients: search by name/phone/email, filter by gender, pagination, CSV export, add/edit/delete
- Patient profile page with tabs for Info, Appointment history, Invoices, and Medical Records

- Medical records per visit (diagnosis, treatment done, prescription, notes) with optional image/PDF attachment upload (X-rays, documents)
- Dentist directory: add/edit/delete profiles with specialization and experience
- Appointments: list view and a full interactive month calendar view; booking, completing, cancelling
- Treatment/service price catalog (admin-managed) that auto-fills item cost when billing
- Billing & invoicing: multi-item invoices, auto total, partial payments, status tracking, and one-click PDF invoice download
- In-app notification bell: new appointments and invoices automatically generate notifications, with unread count and mark-all-read
- Staff management page (admin-only): promote/demote roles, activate/deactivate, remove staff, with a safeguard against self-deletion
- Dark mode toggle (persisted), toast notifications for every create/update/delete action, and a styled 404 page

4. Features Implemented — Patient Portal

- Independent patient registration & login with its own JWT session, kept fully separate from staff logins
- Smart registration: if the clinic staff already has this person on file, registering links portal access to that existing record instead of duplicating it
- Patient dashboard: next appointment, completed visit count, outstanding balance, medical report count
- Read-only appointment history, scoped so a patient can only ever see their own appointments
- Payment history with a full itemized breakdown per invoice
- Medical report history (diagnosis, treatment, prescription, notes per visit)
- One-click PDF download for both payment receipts and medical reports, generated server-side with PDFKit
- "Email me" buttons to re-send a receipt or report on demand, with an honest status message if email isn't configured
- Automatic emails: welcome email on registration, booking confirmation when an appointment is created, a payment receipt (PDF attached) when an invoice is created, and a reminder email roughly a day before an upcoming appointment (checked hourly by a background job)

5. Suggested Features (Not Yet Built)

- SMS appointment reminders alongside email (would need a provider like Twilio)
- Real password-reset-by-email flow for both staff and patients (currently change-password requires being logged in)
- Email verification (click-to-confirm) before a new patient account is fully activated
- Recurring/multi-slot appointment booking with dentist availability conflict checks
- Letting patients request/book their own appointments directly (currently the portal is read-only; booking is still done by staff)
- Audit log of who changed what and when

6. Folder Structure

backend/ — Express server, Mongoose models (User, Patient, Dentist, Appointment, Invoice, Service, MedicalRecord, Notification), route files per resource including `patientAuth.js` and `portal.js` for the patient portal, JWT auth + role middleware (including patient-only), Multer upload config, a mailer + email templates + PDF generator under `utils/config`, an hourly reminder cron job, and an `uploads/` folder for stored files.

frontend/ — React app (Vite): staff pages (Landing, Login, Register, Dashboard, Patients, PatientDetail, Dentists, Appointments, Billing, Services, Staff, Profile, NotFound) plus a separate patient-portal page set (PatientLogin, PatientRegister, PatientDashboard, PatientAppointments, PatientPayments, PatientRecords) with its own layout, sidebar, Axios instance, and auth context so the two portals never share a login session.

7. How To Run It

Backend: `cd backend → npm install → copy .env.example to .env` and set your MongoDB connection string (SMTP variables are optional — leave blank to run without email) → `npm run dev` (runs on port 5000).

Frontend: `cd frontend → npm install → copy .env.example to .env → npm run dev` (runs on port 5173). Register a staff account first (pick 'admin' as the role to unlock Staff Management and pricing controls), then log in. Patients use the separate `/patient-login` area.

8. Testing Done

Backend: every file passed Node syntax checks, all dependencies installed cleanly, and the server was booted and its endpoints verified live with curl — every protected route (staff and patient portal alike) correctly returned 401 Unauthorized rather than 404, confirming the routes and their auth middleware are properly wired. The mailer was verified to fail gracefully with no SMTP configured (returns false and logs a warning rather than crashing). Frontend: dependencies installed, a production build (vite build) completed with zero errors, and both servers were run together and confirmed serving correctly, including the new `/patient-login` route. Full end-to-end testing with live data (and actually sending emails) requires a running MongoDB instance and real SMTP credentials, which this sandbox environment cannot provision on its own — connect both via the backend `.env` file to exercise the complete booking/billing/records/email flow.